Layouts

Layouts are the structure of an android application

LinearLayout

orientation=”vertical” or horizontal

RelativeLayout

layout_centerHorizontal

layout_centerVertical

layout_centerInParent

layout_above

layout_below

layout_toRightOf

layout_toLeftOf

layout_alignParentTop

layout_alignParentBottom

FrameLayout

TableLayout

Arrange Views in rows and columns

<TableRow>

</TableRow>

UI Controls

TextView

TextView displays text to the user and optionally allows them to edit it programmatically. TextViewis a complete text editor, however basic class is configured to not allow editing but we can edit it.

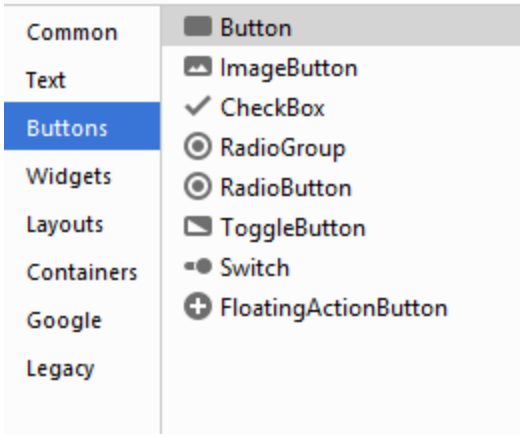
View is the parent class of TextView. Being a subclass of view the text view component can be used in your app's GUI inside a ViewGroup, or as the content view of an activity.
We can create a TextView instance by declaring it inside a layout(XML file) or by instantiating it programmatically(Java Class).

EditText

EditText is a standard entry widget in android apps. It is an overlay over TextView that configures itself to be editable. EditText is a subclass of TextView with text editing operations. **We often use EditText in our applications in order to provide an input or text field, especially in forms.** The most simple example of EditText is Login or Sign-in form.

Button

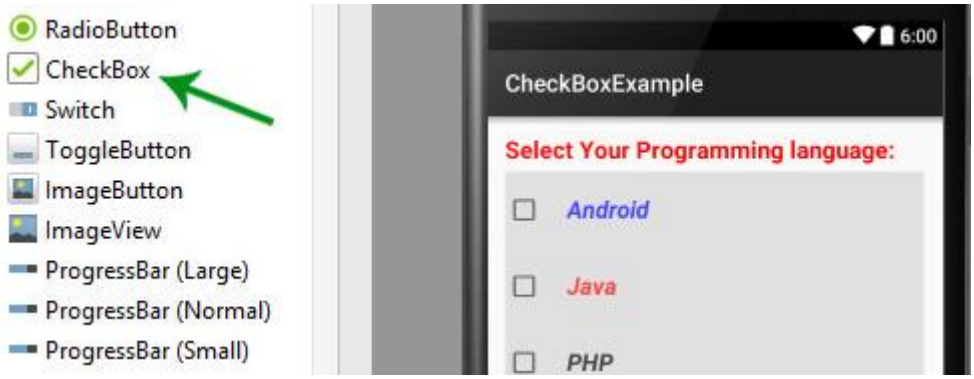
Button represents a push button. A Push buttons can be clicked, or pressed by the user to perform an action. There are different types of buttons used in android such as CompoundButton, ToggleButton, RadioButton.



Button is a subclass of TextView class and compound button is the subclass of Button class. **On a button we can perform different actions or events like click event, pressed event, touch event etc.**
Android buttons are GUI components which are sensible to taps (clicks) by the user. *When the user taps/clicks on button in an Android app, the app can respond to the click/tap*. These buttons can be divided into two categories: the first is Buttons with text on, and second is buttons with an image on. A button with images on can contain both an image and a text. Android buttons with images on are also called *ImageButton*.

CheckBox

CheckBox is a type of two state button either unchecked or checked in Android. Or you can say it is a type of on/off switch that can be toggled by the users. You should use checkbox when presenting a group of selectable options to users that are not mutually exclusive. CompoundButton is the parent class of CheckBox class.



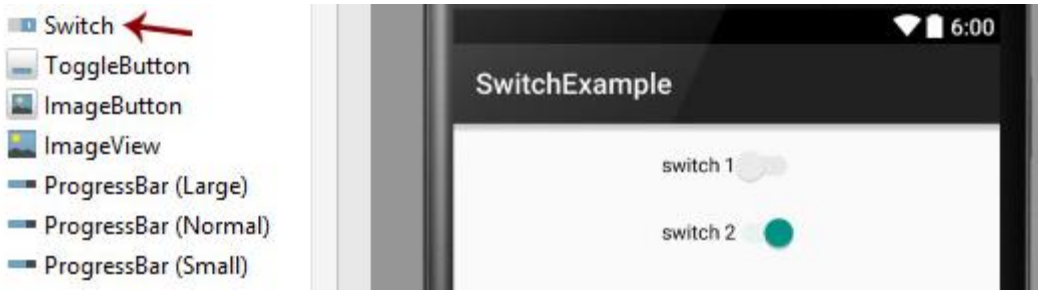
In android there is a lot of usage of check box. For example, to take survey in Android app we can list few options and allow user to choose using CheckBox. The user will simply checked these checkboxes rather than type their own option in EditText. Another very common use of CheckBox is as remember me option in Login form.

ImageView

ImageView class is used to display an image file in application. Image file is easy to use but hard to master in Android, because of the various screen sizes in Android devices. An android is enriched with some of the best UI design widgets that allows us to build good looking and attractive UI based application.

Switch

Switch is a two-state toggle switch widget that can select between two options. It is used to display checked and unchecked state of a button providing slider control to user. Switch is a subclass of CompoundButton. It is basically an off/on button which indicate the current state of Switch. It is commonly used in selecting on/off in Sound, Bluetooth, WiFi etc.



As compared to ToggleButton Switch provides user slider control. The user can simply tap on a switch to change its current state.
Switch allow the users to change the setting between two states like turn on/off wifi, Bluetooth etc from your phone's setting menu. It was introduced after Android 4.0 version (API level 14).

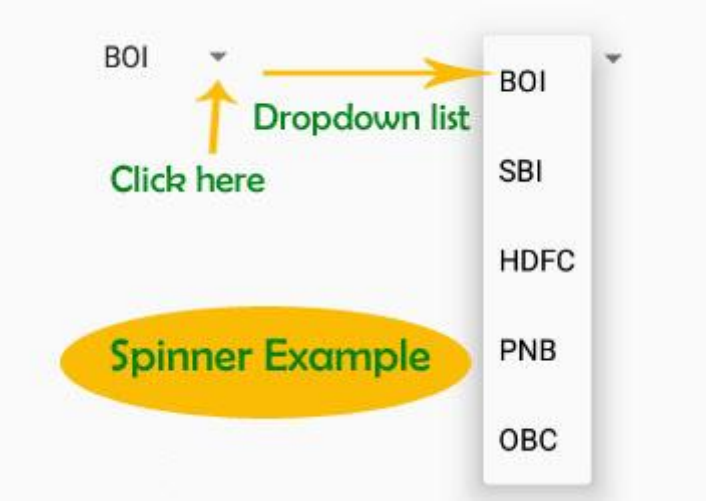
RadioButton and ButtonGroup

RadioButton are mainly used together in a RadioGroup. In RadioGroup checking the one radio button out of several radio button added in it will automatically unchecked all the others. It means at one time we can checked only one radio button from a group of radio buttons which belong to same radio group. The most common use of radio button is in Quiz Android App code.

RadioButon is a two state button that can be checked or unchecked. If a radio button is unchecked then a user can check it by simply clicking on it. Once a RadiaButton is checked by user it can't be unchecked by simply pressing on the same button. It will automatically unchecked when you press any other RadioButton within same RadioGroup.

Spinner

Spinner provides a quick way to select one value from a set of values. Android spinners are nothing but the drop down-list seen in other programming languages. In a default state, a spinner shows its currently selected value. It provides easy way to select a value from a list of values.



In Simple Words we can say that a spinner is like a combo box of AWT or swing where we can select a particular item from a list of items. Spinner is a sub class of AsbSpinner class.

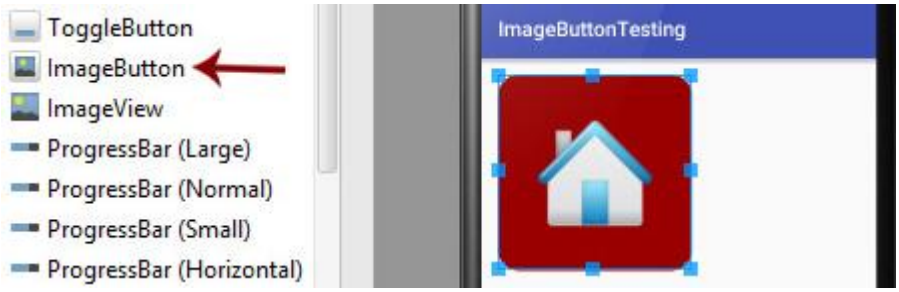
```
<Spinner
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:id="@+id/five"
    android:entries="@array/country_arrays"></Spinner>
```

Write the following array code inside string.xml file

```
<string-array name="country_arrays">
    <item>Napal</item>
    <item>United States</item>
    <item>Indonesia</item>
    <item>France</item>
    <item>Italy</item>
    <item>Singapore</item>
    <item>New Zealand</item>
    <item>India</item>
</string-array>
```

ImageButton

ImageButton is used to display a normal button with a custom image in a button. In simple words we can say, ImageButton is a button with an image that can be pressed or clicked by the users. By default it looks like a normal button with the standard button background that changes the color during different button states.

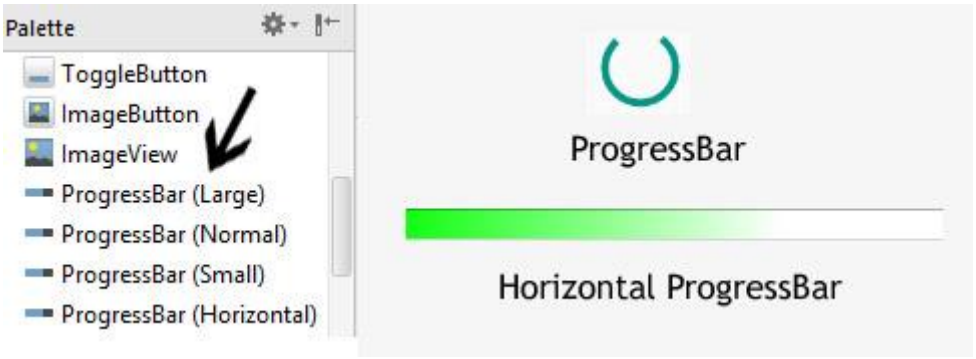


An image on the surface of a button is defined within a xml (i.e. layout) by using src attribute or within java class by using setImageResource() method. We can also set an image or custom drawable in the background of the image button.

ToggleButton

PrograssBar

ProgressBar is used to display the status of work being done like analyzing status of work or downloading a file etc. In Android, by default a progress bar will be displayed as a spinning wheel but If we want it to be displayed as a horizontal bar then we need to use style attribute as horizontal. It mainly use the **"android.widget.ProgressBar"** class.



TimePicker

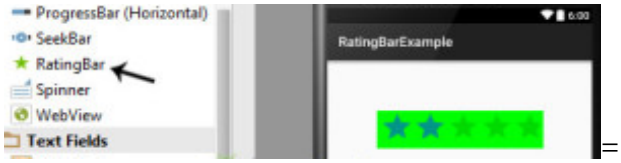
TimePicker is a widget used for selecting the time of the day in either AM/PM mode or 24 hours mode. The displayed time consist of hours, minutes and clock format. If we need to show this view as a Dialog then we have to use a TimePickerDialog class.

DatePicker

DatePicker is a widget used to select a date. It allows to select date by day, month and year in your custom UI (user interface). If we need to show this view as a dialog then we have to use a DatePickerDialog class. For selecting time Android also provides timepicker to select time.

RatingBar

RatingBar is used to get the rating from the app user. A user can simply touch, drag or click on the stars to set the rating value. The value of rating always returns a floating point number which may be 1.0, 2.5, 4.5 etc.



=

In Android, RatingBar is an extension of ProgressBar and SeekBar which shows a rating in stars. RatingBar is a subclass of AbsSeekBar class.

WebView

WebView is a view used to display the web pages in application. This class is the basis upon which you can roll your own web browser or simply use it to display some online content within your Activity. We can also specify HTML string and can show it inside our application using a WebView. Basically, WebView turns application into a web application.

Webpage in WebView

Static HTML in WebView

In order to add Web View in your application, you have to add **<WebView>** element to your XML(layout) file or you can also add it in java class.

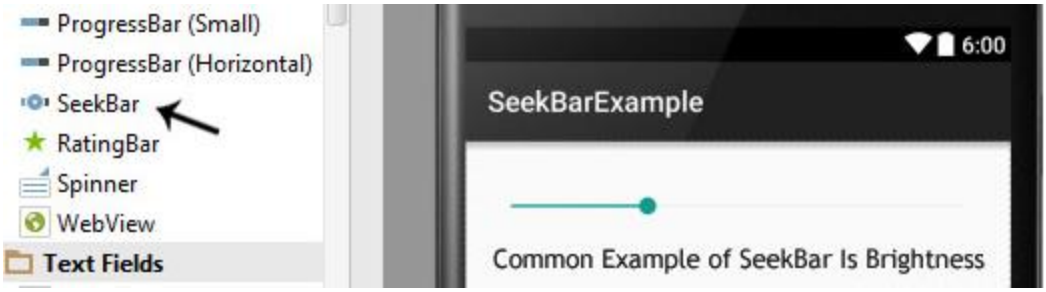
CalenderView

Calendar View widget was added in API level 11(Android version 3.0) which means this view is only supported in the device that are running on Android 3.0 and higher version. It is used for displaying and selecting dates.

The supported range of dates of this calendar is configurable. User can select a date by taping/clicking on it and also can scroll & find the calendar to a desired date. Developer can also set minimum and maximum date shown in calendar view.

SeekBar

SeekBar is an extension of ProgressBar that adds a draggable thumb, a user can touch the thumb and drag left or right to set the value for current progress.



SeekBar is one of the very useful user interface element in Android that allows the selection of integer values using a natural user interface. An example of SeekBar is your device's brightness control and volume control.

layout file

ListView Control

layout file

```
<ListView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/lvLanguage">
</ListView>
```

Activity file

```
final String[] languages = {"C", "C++", "Java", "PHP", "C#", "Kotlin"};
ListView lvLanguage = findViewById(R.id.lvLanguage);

ArrayAdapter<String> adapter = new ArrayAdapter<>(getApplicationContext(),
android.R.layout.simple_list_item_1, languages);
lvLanguage.setAdapter(adapter);

lvLanguage.setOnItemClickListener(new AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(AdapterView<?> adapterView, View view, int i, long l) {
        Toast.makeText(getApplicationContext(), languages[i],
Toast.LENGTH_LONG).show();
    }
});
```

ListView and Adapter

String[] NAME = {"C", "C++", "VB", "PHP", "JAVA", "C#", "Android"};

ListView lvProgrammingLanguage = (ListView) findViewById(R.id.lvProgrammingLanguage);

ArrayAdapter<String> adapter = new ArrayAdapter<String>(getApplicationContext(), android.R.layout.simple_list_item_1, NAME);

```
        lvProgrammingLanguage.setAdapter(adapter);
```

Custom ListView

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:orientation="vertical" android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ImageView
        android:id="@+id/ivImage"
        android:layout_width="50dp"
        android:layout_height="50dp"
        android:layout_alignParentLeft="true"
        android:layout_alignParentStart="true"
        android:layout_alignParentTop="true"
        android:layout_marginLeft="39dp"
        android:layout_marginStart="39dp"
        android:layout_marginTop="32dp"
        app:srcCompat="@mipmap/ic_launcher" />

    <TextView
        android:id="@+id/tvName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginLeft="18dp"
        android:layout_marginStart="18dp"
```

```
    android:textStyle="bold"
    android:textAppearance="@style/TextAppearance.AppCompat.Large"
    android:text="Name"
    android:layout_alignTop="@+id/ivImage"
    android:layout_toRightOf="@+id/ivImage"
    android:layout_toEndOf="@+id/ivImage" />

  <TextView
    android:id="@+id/tvDescription"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Description"

android:textAppearance="@style/Base.TextAppearance.AppCompat.Light.Widget.PopupMenu.Small"
    android:textStyle="italic"
    android:layout_alignBottom="@+id/ivImage"
    android:layout_alignLeft="@+id/tvName"
    android:layout_alignStart="@+id/tvName" />
</RelativeLayout>



  private static String[] NAME = {"C", "C++", "VB", "PHP", "JAVA", "C#", "Android"};
  private static int[]
IMAGE={R.drawable.c,R.drawable.cplus,R.drawable.csharp,R.drawable.php,R.drawable.java,R.drawable.
csharp,R.drawable.java};
  private static String[] DESCRIPTION ={"Procedural Programming","Object Oriented
Programming","Visual Basic ","Hypertext Preprocessor","Object Oriented Programming","C
sharp","Android App"};


    ListView lvProgrammingLanguage = (ListView) findViewById(R.id.lvProgrammingLanguage);
```

```
    CustomAdapter adapter = new CustomAdapter();
    lvProgrammingLanguage.setAdapter(adapter);


private class CustomAdapter extends BaseAdapter {

    @Override
    public int getCount() {
       return NAME.length;
    }

    @Override
    public Object getItem(int i) {
       return null;
    }

    @Override
    public long getItemId(int i) {
       return 0;
    }

    @Override
    public View getView(int i, View view, ViewGroup viewGroup) {
       view = getLayoutInflater().inflate(R.layout.custom_layout, null);

       ImageView ivImage = (ImageView) view.findViewById(R.id.ivImage);
       TextView tvName = (TextView) view.findViewById(R.id.tvName);
       TextView tvDescription = (TextView) view.findViewById(R.id.tvDescription);
```

```
        ivImage.setImageResource(IMAGE[i]);
        tvName.setText(NAME[i]);
        tvDescription.setText(DESCRIPTION[i]);

        return view;
      }
   }
```

Toast

```
                Context context = getApplicationContext();
            String message = "Toast Message ";
            int duration = Toast.LENGTH_LONG;

            Toast tost = Toast.makeText(context, message, duration);

            tost.setGravity(Gravity.BOTTOM, 0, 0);
            tost.show();
```

Notification

```
  private NotificationCompat.Builder notification;
  private static final int uniqueId = 12345;


          notification = new NotificationCompat.Builder(this);
      notification.setAutoCancel(true);
```

```
                notification.setSmallIcon(R.mipmap.ic_launcher);
        notification.setTicker("I am ticker Text");
        notification.setContentTitle("Title of the Notification");
        notification.setContentText("I am body of the notificaiton ");
        notification.setWhen(System.currentTimeMillis());

        Intent intent = new Intent(getApplicationContext(), MainActivity.class);
        PendingIntent pendingIntent = PendingIntent.getActivity(getApplicationContext(), 0, intent,
PendingIntent.FLAG_UPDATE_CURRENT);
        notification.setContentIntent(pendingIntent);

        NotificationManager notificationManager = (NotificationManager)
getSystemService(NOTIFICATION_SERVICE);
        notificationManager.notify(uniqueId, notification.build());
```

AsyncTask Example

```
package np.com.sunilbist.helloworld;

import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.Context;
import android.content.Intent;
import android.os.AsyncTask;
import android.support.v4.app.NotificationCompat;
import android.support.v4.widget.TextViewCompat;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.view.Gravity;
```

```
import android.view.View;
import android.view.ViewGroup;
import android.view.Window;
import android.widget.ArrayAdapter;
import android.widget.BaseAdapter;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.ListView;
import android.widget.TextView;
import android.widget.Toast;

import java.util.ArrayList;

public class AsyncActivity extends AppCompatActivity {

   String[] NAME = {"C", "C++", "VB", "PHP", "JAVA", "C#", "Android"};

   ListView lvProgrammingLanguage;

   @Override
   protected void onCreate(Bundle savedInstanceState) {
      super.onCreate(savedInstanceState);
      setContentView(R.layout.activity_main);

      lvProgrammingLanguage = (ListView) findViewById(R.id.lvProgrammingLanguage);
      ArrayAdapter<String> adapter = new ArrayAdapter<String>(getApplicationContext(),
android.R.layout.simple_list_item_1, new ArrayList<String>());
      lvProgrammingLanguage.setAdapter(adapter);
      new MyTask().execute();
```

```
}


class MyTask extends AsyncTask<Void, String, Void> {
    ArrayAdapter<String> adapter;

    @Override
    protected void onPreExecute() {
        adapter = (ArrayAdapter<String>) lvProgrammingLanguage.getAdapter();
    }

    @Override
    protected Void doInBackground(Void... voids) {
        for(String s : NAME){
            publishProgress(s);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        return null;
    }

    @Override
    protected void onProgressUpdate(String... values) {

        adapter.add(values[0]);
    }
```

```
        @Override
        protected void onPostExecute(Void aVoid) {
            Toast.makeText(getApplicationContext(), "All items are added Successfully",
Toast.LENGTH_LONG).show();
        }
    }
}
```
